

OPAUDIT 017 — One comment, 277,000 deliveries

Audit ID	2026-09-15-017
Report date	2026-09-15
First observed	2026-09-14 evening, on the host’s Proxmox graphs: CPU bursting to 30–45% every ten minutes, load 6–7 on eight cores, the fan on constantly. “I’ve never had the box work this hard and this often before.”
Theme	Every SnapSmack site names its posts the same way. A reply to <i>someone else’s</i> image 388 looked, to every site in the fleet, like a reply to <i>its own</i> image 388 — so every site filed it, and every site forwarded it to everyone. Sean: “microphone audio feedback, fediverse version.”
Surfaces	The inbox (replies, likes, boosts, undo, delete); the 712D inbox-forwarding rule; the delivery queue; the public comment thread on 27 sites; the shared MariaDB box.
State	WATCHED WORKING 0.7.718D deployed fleet-wide 2026-09-15; queues, misfiled comments, stray boosts and log cleaned on both boxes; six consecutive quiet ticks watched on the host graphs (CPU 2–4%, load under 1).
Watched live?	Yes, both ways. The defect: the queue contents were read and every row was the same Note. The fix: the graph, before and after, on the same box.
Reporter	Sean (read the graphs, refused “it’s just the new cron”, kept asking) + Claude (wrote the bug in 712D, found it, fixed it).
Related	OPAUDIT 014 (the fleet cron finally running is what made this visible); OPAUDIT 015 (712D inbox forwarding, “built and tested” — tested on the mechanism, never on a reply to someone else’s post); OPAUDIT 013/708D (comments federating at all).
Publication	Public. No exploitable detail. One user (Sean) runs every affected site.

1. SUMMARY

A single comment — #1 on usedcarparts.photoblogs.fyi, a reply to that site’s image 388 — was delivered to the site’s followers, as it should be. Each follower is another SnapSmack site. Each of them resolved the reply’s target, `/ap/note/i/388`, by pattern alone, without checking the host, and concluded it was a reply to *its own* image 388. So each filed it as an approved, public comment on the wrong photograph, and — per the inbox forwarding rule added in 712D — forwarded it to all of *its* followers as “a reply to our post”. Those followers did the same. Nearly every site follows nearly every other, so the same Note kept arriving from a new direction, and every arrival queued 25 more sends. `acolourlesslife.ca` held 68,131 copies; the fleet about 277,000 on the first count, refilling within one tick of being dumped. The box spent four minutes of every ten, per site, sending one comment to itself.

2. WHAT A HUMAN SAW

When	What
------	------

2026-09-14, 19:30–20:30 Proxmox host: CPU peaks 30–45%, load 6–7, CPU pressure stall 35–45%, IO stall 18%.
Sawtooth every ten minutes. Fan on.

Same night Stale per-site crontab lines removed (OPAUDIT 014’s leftovers); cron.d parallelism 6 → 2.
Peaks halved; plateaus instead of spikes. Fan still on.

716D deployed Hourly roster pull, back-off on unreachable peers, comments kick the worker. No visible change. The tick’s new `TIMING` line named the step: `drain=228s` on three sites, `13s` on the rest, `0.03s` on the one site with nothing queued.

Queue read foundtextures.ca: ~6,000 rows to each of ~25 followers. acolourlesslife ~56,000.
curbappeal ~27,000 — a site with no posts. photofri.day 8,412. Dumped: ~277,000 rows.
Refilled within one tick.

Queue read again Every row on every site: `Create Note`, actor `usedcarparts`, object `/ap/note/c/1`, `inReplyTo /ap/note/i/388`. One Note.

2026-09-15, 10:44–11:44 After 718D + cleanup: CPU 2–4%, load 0.3–0.9, stalls under 1%. Six ticks.

3. ROOT CAUSE — THREE LAYERS

Host-blind object resolution. `sv_resolve_target()` matched `/ap/note/{p|i|c|l}/N` on any URL and returned “image N”. It was written when a site only ever received activities about its own objects. Once sites followed each other, every inbound reply, like, boost, undo and delete about *any* fleet post resolved against local ids. Replies were filed as comments on the wrong photograph; likes and boosts were tallied on the wrong photograph.

Forwarding without ownership. 712D’s inbox forwarding (ActivityPub §7.1.2) is correct as a rule: the server that *owns* a post forwards replies to its followers. It trusted the host-blind resolver to decide ownership, so every site believed it owned every post, and forwarded. And a forwarded copy was itself eligible to be forwarded again, so the hop count was unbounded.

A dense mesh as amplifier. Twenty-five sites, nearly all following nearly all. One hop × 25 × 25 × 25... The relay (photoblogs.fyi) was not the cause; it ran the same code and was caught the same way (16,852 rows).

Underneath: the test that never ran. OPAUDIT 015 called forwarding “built and regression-tested”. The regression checked that a reply to our post is forwarded and that a forwarded copy is verified from its origin. It never sent a reply to *someone else’s* post through the inbox. The one case that mattered in a fleet was the one not exercised.

4. EVIDENCE AT DIAGNOSIS

The `TIMING` line added in 716D, per site, per tick. The queue grouped by host, then by activity type, actor and object: one distinct object across hundreds of thousands of rows, all priority 10 (live publication), all created within one second on the sites that had just received a copy. The inbox code path: `sv_resolve_target` with no host comparison; the forward call gated only on “we kept it”.

5. WHAT WE DID (0.7.718D “FEEDBACK”)

A URL names one of our objects only when it is on our host. `sv_url_is_ours()` now gates every inbound resolve: replies, likes, boosts, undo, delete. A reply to another site’s image 388 is not a reply to ours; it falls through to the mention path like any other remote post.

Only the origin forwards. A copy that reached us as a forward (signer ≠ actor) is never forwarded again. §7.1.2 is one hop from the owner, not a relay chain.

One forward per object per inbox. A dedupe key, so a repeat arrival of the same Note is a no-op rather than another 25 rows.

Box side, after deploy: queues dumped on both boxes (a further ~202,000 rows had refilled); every fediverse comment whose reply target was on another host deleted — 6,000–11,000 per site on the older sites, about 170,000 fleet-wide, all of them other people’s comments shown under the wrong photographs since yesterday’s 715D backfill sent the historic ones; stray boosts of other sites’ posts (0–26 per site) and their notifications removed; all tables rebuilt (MariaDB 2.5 GB → 1.2 GB); the DB container’s buffer pool cut from 2 GB to 1.25 GB on a 4 GB box, which ended its 98% swap.

Also shipped on the way here, each real but not the cause: 716D (hourly roster pull; back-off on unreachable peers; comments kick the worker on post; the TIMING line) and 717D (first-follow catalogue 5 posts, not 200; a failing delivery retried for a week, then dropped).

6. VERIFIED

Regression `tests/fediverse-forward-feedback-regression.php` (11 checks) and the existing fediverse suites pass. Live: the host graphs on 2026-09-15 from 10:44 to 11:44 — six ticks, CPU 2–4%, load under 1, the one 11% blip the hourly rclone backup. Not yet re-verified after the cleanup: that a *genuine* reply to a fleet post still forwards exactly once to the owner’s followers and appears on Pixelfed — OPAUDIT 015’s open item, now with the loop removed underneath it.

7. EYEBALL NEXT

Comment #1 on usedcarparts, watched arriving on pixelfed.social once, and on one fleet site once, after 718D. The HTTP 401s seen between fleet sites on the first queue read (a signature problem, separate). And a fleet-wide cost check: any inbound handler that resolves an id by pattern anywhere else in the codebase.

8. THE DOCTRINE THIS INCIDENT CREATED

An id is not an identity. In a fleet where every site mints the same id shapes, nothing about an object is known until its host is checked. And any rule that re-sends what it receives is a feedback loop unless it can prove, each time, that it is the origin.